



**NATIONAL UNIVERSITIES OF MODERN
LANGUAGES
FACULTY OF ENGINEERING AND COMPUTING
DEPARTMENT OF SOFTWARE ENGINEERING**



To be filled by student

Course Name : Software Quality Engineering _____

Assignment No./Presentation/Project 1_Project _____

Date : 21 May 2026

Submitted by

Class: BS(SE) AFT _____ **Section** _____ **Sesssion (M/A)** _____

Name: Nazish Khan _____ **Roll No.** FL23843 _____

Name: M Abdul Rafay _____ **Roll No.** FL23791 _____

Name: M Ali Atif _____ **Roll No.** FL23824 _____

Submitted to

Instructor Name: Dr. Sumaira Nazir _____

To be filled by Instructor

Instructor Comments:

Marks Obtained: _____ **Total Marks:** _____

CLO No. : _____

Table of Contents

1. Introduction.....	1
1.1 Purpose.....	1
1.2 Scope.....	1
1.3 Definitions and Acronyms	1
1.4 Reference Documents	2
2. Project Description	2
2.1 Project Overview	2
2.2 System Modules.....	2
Patient Module.....	2
Doctor Module.....	3
Admin Module.....	3
2.3 Objectives	3
2.4 Stakeholders.....	3
3. Requirement Gathering and Analysis	4
3.1 Functional Requirements	4
3.2 Non-Functional Requirements	5
3.3 Assumptions.....	6
3.4 Constraints	6
4. Software Quality Assurance Management	6
4.1 Quality Objectives	6
4.2 Quality Standards.....	6
4.3 Roles and Responsibilities	7
5. Software Quality Assurance Activities.....	7
Requirement Reviews	7
Design Reviews	7
Code Reviews	8
Testing	8
Audits.....	8
6. Reviews and Audits	8
6.1 Requirement Review.....	8
6.2 Design Review.....	8
6.3 Code Review.....	8
6.4 Quality Audit	8
7. Verification and Validation	9
7.1 Verification	9
7.2 Validation.....	9
8. Software Testing Strategy.....	9
8.1 Unit Testing	9

8.2 Integration Testing	10
8.3 System Testing.....	11
8.4 User Acceptance Testing	11
9. Defect Management.....	11
Defect Severity Levels.....	12
10. Configuration Management	12
11. Risk Management	13
12. Quality Metrics	13
13. Tools and Technologies	14
14. Conclusion	15
15. Source Code.....	16
References.....	17

List of Figures

Figure 1: Use Case Diagram of the Doctor Appointment Booking System.....	4
Figure 2: Software Quality Assurance Process Diagram.....	7
Figure 3: Defect Management Flowchart	11
Figure 4: System Flowchart of the Doctor Appointment Booking System.....	15

List of Tables

Table 1: Definitions and Acronyms.....	2
Table 2: Project Stakeholders	3
Table 3: Roles and Responsibilities.....	7
Table 4: Unit Test Cases.....	9
Table 5: Integration Test Cases	10
Table 6: Defect Severity Levels.....	12
Table 7: Risk Register.....	13
Table 8: Quality Metrics	13
Table 9: Tools and Technologies.....	14

1. Introduction

1.1 Purpose

This Software Quality Assurance Plan (SQAP) has been prepared for the Doctor Appointment Booking System. The purpose of this document is to describe how the development team will ensure that the software is built correctly and meets the required quality standards before it is released to users.

This plan covers all quality-related activities that will take place during the development of this system, including requirement reviews, design reviews, testing, defect management, and audits. It has been prepared in accordance with the IEEE 730:2014 standard, which provides a recognized framework for software quality assurance processes [1].

1.2 Scope

This plan applies to the complete Doctor Appointment Booking System . a web-based application that allows patients to book doctor appointments online. The system supports three types of users: patients, doctors, and administrators, each with their own set of features and access levels.

This SQAP covers all development phases from requirement gathering through to final deployment. All system modules. patient module, doctor module, and admin module. fall within the scope of this plan.

1.3 Definitions and Acronyms

Term	Description
SQAP	Software Quality Assurance Plan
IEEE	Institute of Electrical and Electronics Engineers
SRS	Software Requirements Specification
UAT	User Acceptance Testing
QA	Quality Assurance
UI	User Interface

Term	Description
API	Application Programming Interface

Table 1: Definitions and Acronyms

1.4 Reference Documents

The following documents were referred to during the preparation of this plan:

- IEEE Standard 730:2014 — IEEE Standard for Software Quality Assurance Processes [1]
- Sommerville, I. — Software Engineering, 10th Edition [3]
- Pressman, R.S. — Software Engineering: A Practitioner's Approach, 8th Edition [2]
- System Requirements Document of Doctor Appointment Booking System [5]

2. Project Description

2.1 Project Overview

The Doctor Appointment Booking System is a web-based application designed to simplify the process of booking a doctor appointment for patients. As noted by Sommerville (2016), modern software systems must provide practical solutions to real-world problems, and this system does exactly that by replacing the manual, in-person appointment booking process with a fast and convenient online alternative [3].

In the traditional method, patients have to visit a hospital or clinic physically just to schedule an appointment, which is time-consuming and inconvenient, especially for those who are unwell. This system solves that problem by allowing patients to search for available doctors and book appointments from home.

The system is built using React.js for the frontend, Node.js and Express.js for the backend, and MongoDB as the database.

2.2 System Modules

The system is divided into three main modules:

Patient Module

- Register and log in to the system
- Search for available doctors by name or specialization

- Book an appointment with a selected doctor
- View the current status of a booked appointment

Doctor Module

- Log in to the system
- View all incoming appointment requests from patients
- Approve or reject appointment requests
- Manage personal schedule

Admin Module

- Add, update, or remove doctor profiles
- Manage patient accounts
- Monitor all appointments in the system
- Maintain overall system records

2.3 Objectives

- To provide patients with a simple online way to book doctor appointments
- To reduce the manual workload of appointment management at hospitals
- To give doctors a convenient way to manage their daily schedules
- To maintain organized digital records of all appointments
- To improve the overall experience for both patients and doctors

2.4 Stakeholders

Table 2: Project Stakeholders

Stakeholder	Role
Patients	Book appointments and check appointment status
Doctors	Manage appointments and schedules
Administrator	Manage users and monitor system activity
Development Team	Build and maintain the system

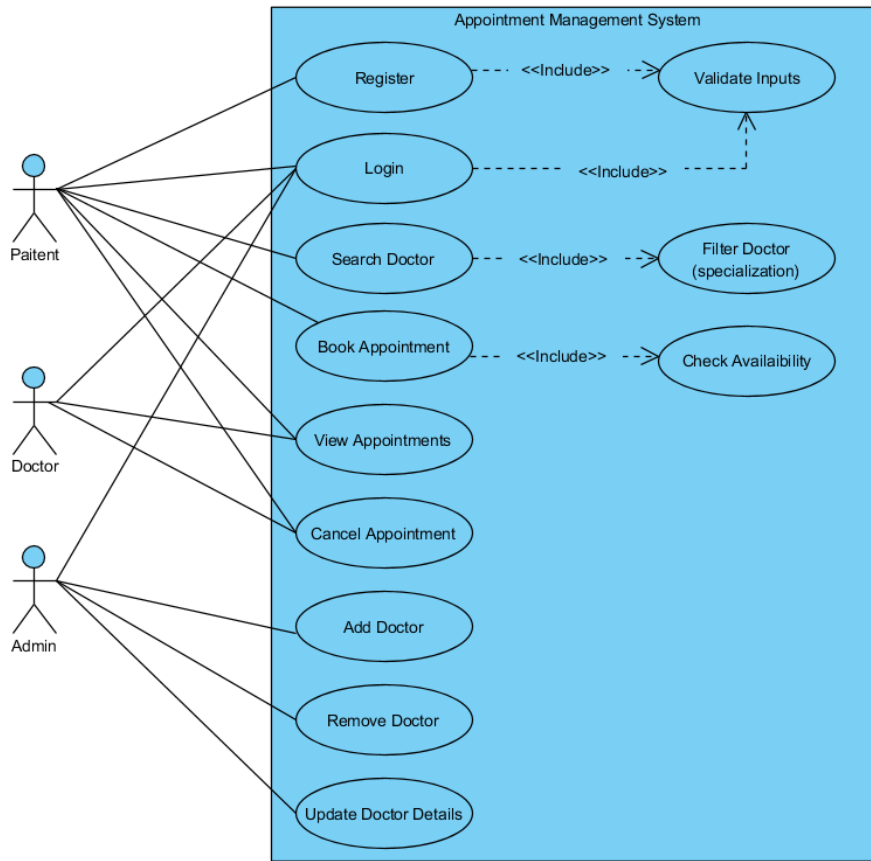


Figure 1: Use Case Diagram of the Doctor Appointment Booking System

3. Requirement Gathering and Analysis

Requirement gathering is the process of identifying and documenting what a system needs to do before development begins. According to Pressman (2014), clear and complete requirements are the foundation of any successful software project. Poor requirement analysis is one of the most common causes of project failure [2].

3.1 Functional Requirements

Functional requirements describe what the system must do. Each requirement below is written using the standard "shall" format used in software engineering practice [2].

FR-01: User Registration: The system shall allow new users to create an account by providing their name, email address, and password.

FR-02: User Login: The system shall allow registered users to log in using their email and password. The system shall verify user credentials before granting access.

FR-03: Doctor Search: The system shall allow patients to search for doctors by name or specialization.

FR-04: Appointment Booking: The system shall allow a logged-in patient to select a doctor, choose an available time slot, and confirm an appointment. The system shall save the appointment details in the database.

FR-05: Appointment Status: The system shall allow patients to view the current status of their booked appointments, including pending, approved, or rejected.

FR-06: Appointment Management by Doctor: The system shall allow doctors to view all incoming appointment requests and approve or reject them.

FR-07: Doctor Management by Admin: The system shall allow the administrator to add new doctors, update doctor profiles, and remove doctors from the system.

FR-08: User Management by Admin: The system shall allow the administrator to view and manage all registered patient accounts.

3.2 Non-Functional Requirements

Non-functional requirements define the quality standards the system must meet. As Sommerville (2016) explains, non-functional requirements describe how well a system performs its functions rather than what those functions are, and they must be measurable to be testable [3].

NFR-01: Performance: The system shall respond to user requests within 3 seconds for at least 95% of all interactions under normal usage conditions.

NFR-02: Reliability: The system shall maintain an uptime of at least 99% per month. Unplanned failures shall not occur more than once per week.

NFR-03: Security: The system shall restrict access to protected pages to authenticated users only. All passwords shall be stored in encrypted form. User sessions shall expire automatically after 24 hours of inactivity.

NFR-04: Usability: The system interface shall be simple enough for a new user to complete the appointment booking process without any external guidance. This will be verified during User Acceptance Testing.

NFR-05: Availability: The system shall be available 24 hours a day, 7 days a week. Planned maintenance shall not exceed 2 hours per month.

NFR-06: Scalability: The system shall support at least 500 concurrent users without any degradation in performance.

NFR-07: Maintainability: The codebase shall be written following clean code practices so that any critical bug can be identified and resolved within 48 hours of being reported.

3.3 Assumptions

- All users have access to a working internet connection
- Users have basic knowledge of how to use a website
- The hosting server will remain operational during normal system usage

3.4 Constraints

- The system requires an active internet connection to function
- Users must be registered and logged in to access any protected feature
- The system depends on the availability of the MongoDB database to operate

4. Software Quality Assurance Management

4.1 Quality Objectives

The following quality objectives have been defined for this project in line with the IEEE 730:2014 standard [1]:

- At least 95% of all identified defects shall be resolved before the final release
- Test cases shall cover at least 85% of all functional requirements
- At least 90% of all executed test cases shall pass during system testing
- At least 85% of participants in User Acceptance Testing shall rate the system as satisfactory
- The system shall have no critical security vulnerabilities at the time of final release

4.2 Quality Standards

The following standards and guidelines will be followed throughout this project:

- IEEE 730:2014 Standard for Software Quality Assurance Processes [1]
- Software testing best practices as described by Pressman (2014) [2]
- Secure coding guidelines to protect user data
- Git branching and version control standards

4.3 Roles and Responsibilities

Table 3: Roles and Responsibilities

Role	Responsibilities
Project Manager	Plan project activities, monitor progress, ensure SQAP is followed
Development Team	Write code, implement requirements, fix reported defects
QA Team	Conduct reviews and audits, verify compliance with standards
Testing Team	Write test cases, execute tests, report and track defects

5. Software Quality Assurance Activities

The following QA activities will be carried out throughout the project lifecycle to ensure quality is maintained at every stage, as required by IEEE 730:2014 [1].

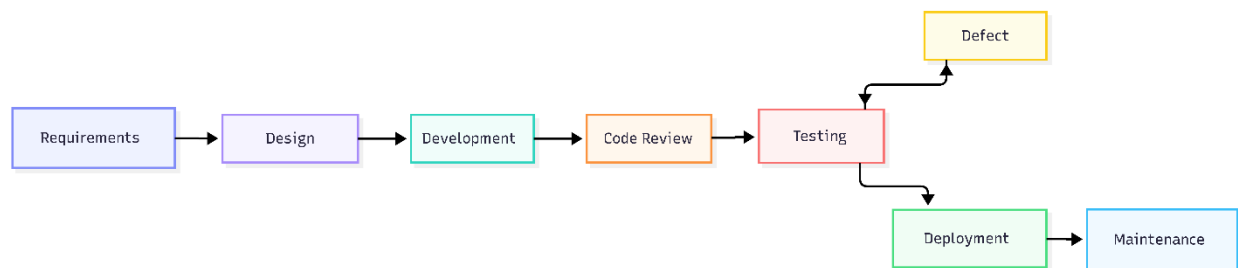


Figure 2: Software Quality Assurance Process Diagram

Requirement Reviews

Before development begins, all requirements will be reviewed by the team to check that they are complete, clear, and free from any contradiction or ambiguity.

Design Reviews

The system design will be reviewed before coding starts. This helps identify design flaws early when they are still inexpensive to fix.

Code Reviews

After writing the code, team members will review each other's work. As Pressman (2014) notes, code reviews are among the most effective methods for detecting defects before testing begins, capable of catching more than 60% of defects early [2].

Testing

All modules will go through unit testing, integration testing, system testing, and User Acceptance Testing as described in Section 8 of this plan.

Audits

At the end of each development phase, a quality audit will be performed to verify that the team is following the standards and processes defined in this plan.

6. Reviews and Audits

6.1 Requirement Review

Requirements will be reviewed before development begins. The purpose is to confirm that all requirements are correctly understood, complete, and consistent with the goals of the project.

6.2 Design Review

The system design — including database structure and module layout — will be reviewed before coding starts to ensure it aligns with the requirements and is technically sound.

6.3 Code Review

Developers will review each other's code after each module is completed. The goal is to catch logical errors, security weaknesses, and any code that does not follow the agreed coding standards.

6.4 Quality Audit

A quality audit will be conducted at the end of each development phase to confirm that all SQAP activities have been properly carried out and that the team is following the IEEE 730:2014 guidelines [1].

7. Verification and Validation

7.1 Verification

Verification is the process of checking that the software has been built correctly according to the requirements and design documents. Sommerville (2016) describes verification as asking the question: "Are we building the product right?" [3].

Verification activities in this project include:

- Reviewing requirement documents for completeness and accuracy
- Reviewing design documents before implementation begins
- Inspecting source code against agreed coding standards
- Reviewing test plans and test cases before execution

7.2 Validation

Validation is the process of checking that the software actually meets the real needs of the users. Sommerville (2016) describes validation as asking: "Are we building the right product?" [3].

Validation activities in this project include:

- Functional testing of all system features
- System testing of the complete application
- User Acceptance Testing with actual users from each user group

8. Software Testing Strategy

Testing is a critical part of software quality assurance. According to Pressman (2014), a proper testing strategy moves from testing individual components to testing the complete system, ensuring that problems are detected as early as possible [2].

8.1 Unit Testing

Unit testing checks each individual module in isolation to verify that it works correctly on its own.

Table 4: Unit Test Cases

Test ID	Module	Test Description	Input	Expected Output
UT-01	Login	Valid login attempt	Correct email and password	User is logged in and taken to dashboard
UT-02	Login	Invalid login attempt	Wrong email or password	Error message is displayed
UT-03	Registration	New user registration	Valid name, email, password	Account is created and saved in database
UT-04	Appointment	Book appointment	Doctor selected, time slot chosen	Appointment is saved in the database
UT-05	Appointment	View appointment status	Logged in patient	Appointment status is displayed correctly

8.2 Integration Testing

Integration testing checks whether different modules work correctly when they are connected and interact with each other.

Table 5: Integration Test Cases

Test ID	Modules Being Tested	Test Description	Expected Result
IT-01	Patient + Appointment	Patient books an appointment	Appointment is created and visible to the doctor
IT-02	Doctor + Appointment	Doctor approves an appointment	Appointment status changes to approved
IT-03	Admin + Doctor	Admin adds a new doctor	New doctor appears in the system and can log in

8.3 System Testing

System testing tests the entire application as a complete product. This stage verifies that all modules work together correctly and that the system meets both functional and non-functional requirements, including performance, security, and usability.

8.4 User Acceptance Testing

User Acceptance Testing (UAT) will be carried out with real users from each user group. patients, doctors, and an administrator. They will be asked to perform specific tasks and provide feedback. The system will be considered to have passed UAT if at least 85% of participants rate it as satisfactory.

9. Defect Management

A defect is any problem in the system that causes it to behave incorrectly or unexpectedly. All defects found during this project will be managed through the process described below, in line with IEEE 730:2014 [1].

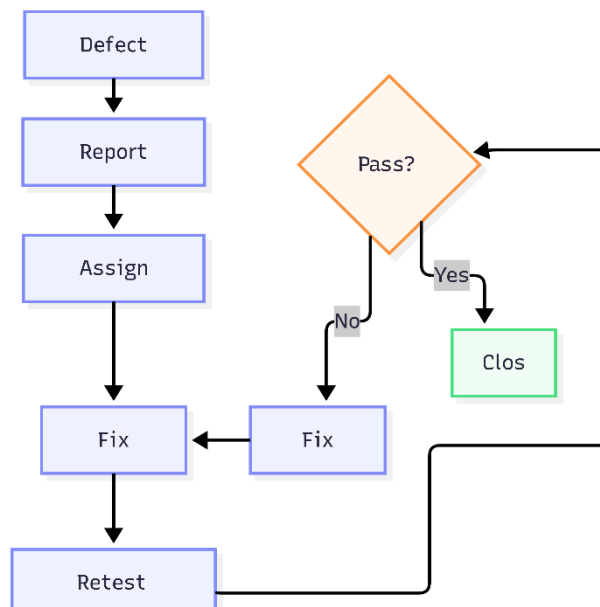


Figure 3: Defect Management Flowchart

1. Defect Identification — A defect is discovered during testing, code review, or system usage.
2. Defect Reporting — The defect is recorded in the defect log with a description, steps to reproduce it, and severity level.

3. Defect Assignment — The defect is assigned to the developer responsible for the affected module.
4. Defect Resolution — The developer fixes the defect and updates the defect log with details of the solution.
5. Retesting — The testing team re-tests the affected area to confirm the fix was successful.
6. Defect Closure — If confirmed fixed, the defect is marked as closed. If the issue remains, it is sent back to the developer.

Defect Severity Levels

Table 6: Defect Severity Levels

Severity	Description
Critical	The system crashes or a major feature is completely broken
Major	A feature does not work correctly but the system still runs
Minor	A small issue that does not affect main functionality
Trivial	A cosmetic issue such as a spelling mistake or alignment problem

10. Configuration Management

Configuration management ensures that all changes to the software and its documents are properly tracked and controlled. This prevents accidental loss of work and ensures the team always has access to the correct and latest version of the code. As Pressman (2014) states, configuration management is essential for maintaining the integrity of a software product throughout its lifecycle [2].

Version Control: Git and GitHub will be used to manage all versions of the source code. Every change will be committed with a clear and descriptive message.

Change Management: Any change to the requirements or source code must be discussed and approved by the project manager before it is implemented.

Release Management: Each released version of the system will be tagged with a version number in GitHub, allowing the team to roll back to any previous version if needed.

Backup Management: The database and all important project files will be backed up every day to prevent data loss.

11. Risk Management

Table 7: Risk Register

Risk	Impact	Probability	Mitigation Strategy
Requirements change during development	High	Medium	Hold regular meetings with stakeholders and document all agreed changes
Development takes longer than planned	Medium	Medium	Review progress every week and adjust tasks if needed
Security weaknesses in the system	High	Low	Follow secure coding practices and test the system for security issues before release
System fails or crashes	High	Low	Test the system regularly and keep updated backups
Important data gets lost	High	Low	Take daily backups of the database and store them separately

12. Quality Metrics

Quality metrics are measurements used to track and evaluate the quality of the software throughout the project. The following metrics will be monitored during development and testing, as recommended by IEEE 730:2014 [1].

Table 8: Quality Metrics

Metric	Description	Target Value
Defect Density	Number of defects found per module during testing	Less than 2 per module
Defect Resolution Rate	Percentage of reported defects that are successfully fixed	At least 95%
Test Coverage	Percentage of functional requirements covered by test cases	At least 85%
Test Pass Rate	Percentage of test cases that pass during execution	At least 90%
User Satisfaction	Percentage of UAT participants who rate the system as satisfactory	At least 85%

13. Tools and Technologies

Table 9: Tools and Technologies

Category	Tool / Technology
Frontend	React.js
Backend	Node.js, Express.js
Database	MongoDB
Version Control	Git, GitHub
Code Editor	Visual Studio Code
Deployment	Vercel

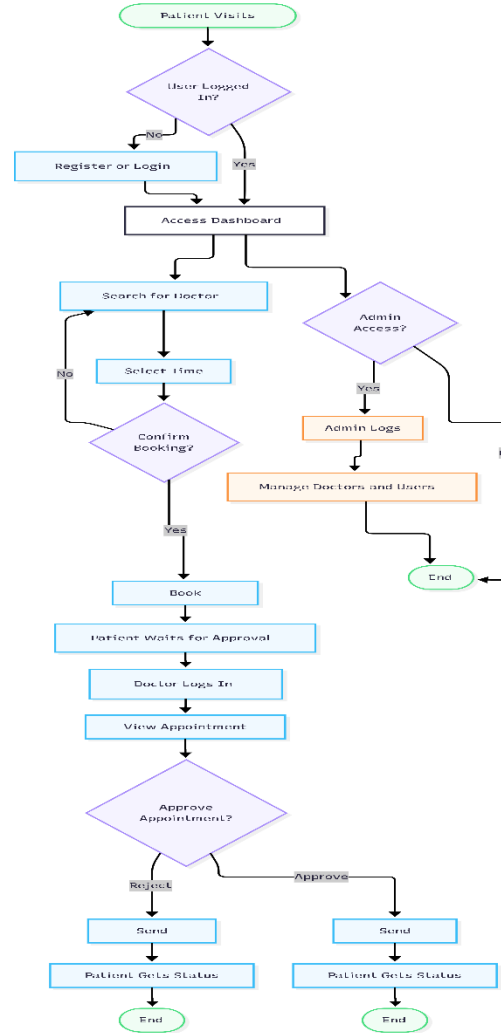


Figure 4: System Flowchart of the Doctor Appointment Booking System

14. Conclusion

This Software Quality Assurance Plan covers all the quality-related activities for the Doctor Appointment Booking System. It has been prepared in accordance with the IEEE 730:2014 standard [1] and includes requirement analysis, testing strategy, defect management, configuration management, risk management, and quality metrics.

By following this plan, the team can ensure that the system is tested properly, defects are resolved on time, and the final product meets the needs of its users. As Pressman (2014) states, software quality does not happen by chance it is the result of a planned, disciplined, and consistent approach to every stage of development [2].

This plan will be followed throughout the entire project to ensure the Doctor Appointment Booking System is delivered as a reliable, secure, and user-friendly product.

15. Source Code

The complete source code for the Doctor Appointment Booking System is available on GitHub at the following repository:

<https://github.com/git-rafaykhan/doctor-oppointment-booking>

The repository contains the full frontend and backend source code, database configuration, and all related project files.

References

- [1] IEEE Standard 730-2014, IEEE Standard for Software Quality Assurance Processes, IEEE, 2014.
- [2] R. S. Pressman, Software Engineering: A Practitioner's Approach, 8th ed., McGraw-Hill Education, 2014.
- [3] I. Sommerville, Software Engineering, 10th ed., Pearson Education, 2016.
- [4] ISO/IEC 25010:2011, Systems and Software Quality Requirements and Evaluation, ISO/IEC, 2011.
- [5] Doctor Appointment Booking System — Project Requirements Document.